



React Native Master Guide — Level 6 (Complete Verbatim Edition)

Global State Management in Mobile: Context API Trap vs Zustand, Atomic Selectors, Persist Middleware & Optimistic UI

EXPO V54 • NATIVEWIND / TAILWIND CSS • ES6+

Level 6.1: The Context API Trap in Mobile vs. Zustand Architecture

1. Mental Model: Why Context API is a Mobile Performance Trap 🐻

In Web apps, small unnecessary component re-renders might go unnoticed because modern desktop CPUs are fast. Kani mobile phones lo, **Context API vaadithe massive UI lag & frame drops vasthayi!**

The React Context Problem:

When you wrap your app in a single `<AppContext.Provider value={{ user, cart, theme, notifications }}>`:

User cart lo oka item add chesinappudu (`cart` state updates)... **Aa context ni subscribe chesina PRATHI OKKA COMPONENT (Profile, Settings, Theme, Navbar) unneeded ga malli re-render avthundi! ❌**

The Zustand Solution (Atomic Selectors):

Zustand lo manam **Selectors** (`useStore(state => state.cartCount)`) vaadatham.

Cart count maarithe... **Cart badge component mathrame re-render avthundi!** Profile, Home Feed, and Settings screens memory lo silent ga untayi. 60 FPS buttery smooth! ✅

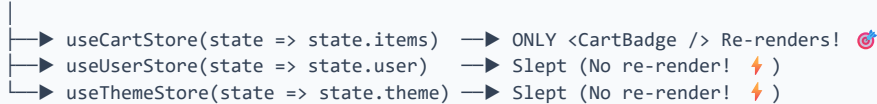
React Context API (Massive Re-render Cascade ❌):

[AppContext: Cart Updated!]



Zustand Atomic Selectors (Zero Waste / Exact Targeted Re-render ✅):

[Zustand Global Store: Cart Updated!]



2. Why Mobile Engineers Prefer Zustand over Redux?

Feature	Redux Toolkit	React Context	Zustand 🐻
Boilerplate Code	High (Actions, Reducers, Dispatchers)	Medium (Providers, Hooks)	Zero (Just <code>create()</code> and use!)
Root Provider Wrapper	Required (<code><Provider></code>)	Required (<code><Context.Provider></code>)	Not Needed (Works anywhere!)
Re-render Optimization	Good (with selectors)	❌ Poor (Re-renders whole subtree)	⚡ Best (Built-in atomic selectors)
Bundle Size	~11 KB	0 KB (Built into React)	~1.1 KB (Ultra lightweight!)

Code Example: Interactive Shopping Cart Store with Atomic Selectors

```
import React, { useState } from 'react';
import { View, Text, Pressable, FlatList } from 'react-native';
import { SafeAreaView, SafeAreaView } from 'react-native-safe-area-context';
import { StatusBar } from 'expo-status-bar';

// =====
// 🐛 1. ZUSTAND-STYLE GLOBAL CART STORE
// =====
let globalStoreState = {
  cart: [
    { id: '1', name: 'AirPods Pro 2', price: 24900, qty: 1 },
  ],
  user: { name: 'Kiran Kumar', tier: 'Prime Gold' },
};

const listeners = new Set();

const cartStore = {
  getState: () => globalStoreState,

  // Action: Add or Increment Item
  addToCart: (product) => {
    const existing = globalStoreState.cart.find((item) => item.id === product.id);
    let updatedCart;

    if (existing) {
      updatedCart = globalStoreState.cart.map((item) =>
        item.id === product.id ? { ...item, qty: item.qty + 1 } : item
      );
    } else {
      updatedCart = [...globalStoreState.cart, { ...product, qty: 1 }];
    }

    globalStoreState = { ...globalStoreState, cart: updatedCart };
    listeners.forEach((listener) => listener());
  },

  // Action: Remove Item
  removeFromCart: (productId) => {
    const updatedCart = globalStoreState.cart
      .map((item) => (item.id === productId ? { ...item, qty: item.qty - 1 } : item))
      .filter((item) => item.qty > 0);

    globalStoreState = { ...globalStoreState, cart: updatedCart };
    listeners.forEach((listener) => listener());
  },

  // Action: Clear Cart
  clearCart: () => {
    globalStoreState = { ...globalStoreState, cart: [] };
    listeners.forEach((listener) => listener());
  },
};

// Custom Hook to Subscribe to Store
const useCartStore = (selector) => {
  const [, forceUpdate] = useState({});

  React.useEffect(() => {
    const unsubscribe = () => listeners.delete(forceUpdate);
    listeners.add(() => forceUpdate({}));
    return () => unsubscribe();
  }, []);

  return selector(cartStore.getState());
};

const CATALOG = [
  { id: '1', name: 'AirPods Pro 2', price: 24900 },
  { id: '2', name: 'Apple Watch Series 10', price: 46900 },
  { id: '3', name: 'MagSafe Battery Pack', price: 10900 },
];
```

```

];

// =====
// 🛒 2. COMPONENT A: Header Cart Badge (Atomic Selector)
// =====
const HeaderCartBadge = () => {
  // Selects ONLY total items count
  const totalItems = useCartStore((state) =>
    state.cart.reduce((sum, item) => sum + item.qty, 0)
  );

  return (
    <View className="flex-row justify-between items-center bg-slate-900 border border-slate-800 p-4 rounded-2xl mb-4">
      <View>
        <Text className="text-white font-extrabold text-lg">Zustand Mobile Store 🐱</Text>
        <Text className="text-slate-400 text-xs">Atomic Global State Management</Text>
      </View>

      <View className="bg-sky-500/20 border border-sky-500/30 px-3.5 py-2 rounded-xl flex-row items-center gap-2">
        <Text className="text-base">🛒</Text>
        <Text className="text-sky-400 font-black text-sm">{totalItems} items</Text>
      </View>
    </View>
  );
};

// =====
// 🏠 3. MAIN APP
// =====
const App = () => {
  const cart = useCartStore((state) => state.cart);
  const totalPrice = useCartStore((state) =>
    state.cart.reduce((sum, item) => sum + item.price * item.qty, 0)
  );

  return (
    <SafeAreaView>
      <StatusBar style="light" backgroundColor="#090d16" />
      <SafeAreaView className="flex-1 bg-slate-950 px-5 pt-3 justify-between pb-6">

        <View className="flex-1">
          <HeaderCartBadge />

          {/* Product Catalog */}
          <Text className="text-white font-bold text-sm mb-3">Available Catalog</Text>
          <View className="gap-2.5 mb-6">
            {CATALOG.map((item) => (
              <View
                key={item.id}
                className="bg-slate-900 border border-slate-800 p-3.5 rounded-2xl flex-row justify-between items-center"
              >
                <View>
                  <Text className="text-white font-bold text-sm">{item.name}</Text>
                  <Text className="text-emerald-400 font-extrabold text-xs mt-0.5">
                    ₹{item.price.toLocaleString('en-IN')}
                  </Text>
                </View>

                <Pressable
                  onPress={() => cartStore.addToCart(item)}
                  className="bg-sky-500 active:bg-sky-600 px-4 py-2 rounded-xl"
                >
                  <Text className="text-white font-bold text-xs">+ Add</Text>
                </Pressable>
              </View>
            ))}
          </View>

          {/* Live Cart Breakdown */}
          <View className="flex-row justify-between items-center mb-2">
            <Text className="text-white font-bold text-sm">Live Cart Items</Text>
            {cart.length > 0 && (
              <Pressable onPress={() => cartStore.clearCart()} hitSlop={10}>

```

```

        <Text className="text-red-400 font-bold text-xs">Clear Cart</Text>
      </Pressable>
    )}
  </View>

  {cart.length === 0 ? (
    <View className="bg-slate-900/50 border border-slate-800/80 p-6 rounded-2xl items-center justify-center">
      <Text className="text-slate-500 text-xs font-semibold">Your cart is currently empty 🛒</Text>
    </View>
  ) : (
    <FlatList
      data={cart}
      keyExtractor={(item) => item.id}
      ItemSeparatorComponent={() => <View className="h-2" />}
      renderItem={({ item }) => (
        <View className="bg-slate-900 border border-slate-800 p-3 rounded-xl flex-row justify-between items-center">
          <View className="flex-1 pr-2">
            <Text className="text-slate-200 font-bold text-xs">{item.name}</Text>
            <Text className="text-slate-400 text-[11px]">
              ₹{item.price.toLocaleString('en-IN')} × {item.qty}
            </Text>
          </View>
          <View className="flex-row items-center gap-2">
            <Pressable
              onPress={() => cartStore.removeFromCart(item.id)}
              className="w-7 h-7 bg-slate-800 rounded-lg items-center justify-center active:bg-red-500/20"
            >
              <Text className="text-white font-bold text-xs">-</Text>
            </Pressable>
            <Text className="text-sky-400 font-bold text-xs">{item.qty}</Text>
            <Pressable
              onPress={() => cartStore.addToCart(item)}
              className="w-7 h-7 bg-slate-800 rounded-lg items-center justify-center active:bg-emerald-500/20"
            >
              <Text className="text-white font-bold text-xs">+</Text>
            </Pressable>
          </View>
        </View>
      )}
    </View>
  )}
</View>

  { /* Checkout Summary Bar */
    <View className="bg-slate-900 border border-slate-800 p-4 rounded-2xl flex-row justify-between items-center shadow-2xl">
      <View>
        <Text className="text-slate-400 text-xs">Total Amount:</Text>
        <Text className="text-emerald-400 text-2xl font-black">
          ₹{totalPrice.toLocaleString('en-IN')}
        </Text>
      </View>
      <Pressable
        disabled={cart.length === 0}
        onPress={() => alert(`Proceeding to checkout with total: ₹${totalPrice.toLocaleString('en-IN')}`)}
        className={`px-6 py-3.5 rounded-xl ${
          cart.length > 0 ? 'bg-emerald-500 active:bg-emerald-600' : 'bg-slate-800 opacity-50'
        }}
      >
        <Text className="text-slate-950 font-black text-xs">Checkout 🛒</Text>
      </Pressable>
    </View>

  </SafeAreaView>
</SafeAreaProvider>

);
};

```

```
export default App;
```

Telugu + English Explanation:

- **Zustand vs Context API:** React Context lo data change ayithe aa Context kinda unna anni components unnecessary ga re-render avthayi. Kani Zustand lo `useCartStore((state) => state.cart)` ani exact ga  data kavalano adi mathrame theskuntam. Deeni valla target component mathrame re-render avthundi.
- **No Provider Boilerplate:** Directly ye file lo kavalante aa file lo store import cheskuni state read or modify cheyochu.

Level 6.2 & 6.3: Zustand Store Architecture, persist Middleware & Async Actions

1. Mental Model: The create() Function

In production apps, we create stores using Zustand's `create()` function with `set` and `get` parameters for atomic updates and async action calls.

2. Persistent Stores with Middleware (persist + AsyncStorage / MMKV)

When a user closes your app, all in-memory state is wiped clean. Zustand's `persist` middleware automatically writes state to disk and re-hydrates (restores) state when the app reopens!

Zustand State Change → Automatically writes to AsyncStorage / MMKV 
App Cold Restart  → Zustand re-hydrates (restores) state before first render!

Code Example: Complete Persistent Theme & Wallet Store

```
import React, { useState, useEffect } from 'react';
import { View, Text, Pressable, FlatList, TextInput } from 'react-native';
import { SafeAreaView, SafeAreaView } from 'react-native-safe-area-context';
import { StatusBar } from 'expo-status-bar';

// 📁 1. MOCK ASYNC STORAGE ENGINE
let mockStorageDisk = {
  theme: 'dark',
  walletBalance: 2500,
  transactions: [
    { id: '1', title: 'Salary Bonus', amount: 5000, type: 'credit', time: 'Yesterday' },
    { id: '2', title: 'Swiggy Order', amount: 450, type: 'debit', time: 'Today' },
  ],
};

const StorageEngine = {
  getItem: async (key) => {
    await new Promise((res) => setTimeout(res, 200));
    return mockStorageDisk[key];
  },
  setItem: async (key, value) => {
    mockStorageDisk[key] = value;
  },
};

// 📁 2. ZUSTAND GLOBAL PERSISTENT WALLET STORE
let walletState = {
  theme: 'dark',
  walletBalance: 2500,
  transactions: [
    { id: '1', title: 'Salary Bonus', amount: 5000, type: 'credit', time: 'Yesterday' },
    { id: '2', title: 'Swiggy Order', amount: 450, type: 'debit', time: 'Today' },
  ],
  isProcessing: false,
};

const walletListeners = new Set();

const walletStore = {
  getState: () => walletState,

  // Action 1: Toggle Theme
  toggleTheme: () => {
    const nextTheme = walletState.theme === 'dark' ? 'light' : 'dark';
    walletState = { ...walletState, theme: nextTheme };
    StorageEngine.setItem('theme', nextTheme);
    walletListeners.forEach((l) => l());
  },

  // Action 2: Async Wallet Recharge
  addFunds: async (amount) => {
    walletState = { ...walletState, isProcessing: true };
    walletListeners.forEach((l) => l());

    await new Promise((res) => setTimeout(res, 1200));

    const newTx = {
      id: Date.now().toString(),
      title: 'UPI Wallet Top-up',
      amount: amount,
      type: 'credit',
      time: 'Just now',
    };

    walletState = {
      ...walletState,
      walletBalance: walletState.walletBalance + amount,
      transactions: [newTx, ...walletState.transactions],
      isProcessing: false,
    };
  },
};
```

```

StorageEngine.setItem('walletBalance', walletState.walletBalance);
StorageEngine.setItem('transactions', walletState.transactions);
walletListeners.forEach((l) => l());
},

// Action 3: Transfer Funds (Debit)
sendPayment: async (recipient, amount) => {
  if (walletState.walletBalance < amount) {
    alert('Insufficient wallet balance! ⚠️');
    return;
  }

  walletState = { ...walletState, isProcessing: true };
  walletListeners.forEach((l) => l());

  await new Promise((res) => setTimeout(res, 1000));

  const newTx = {
    id: Date.now().toString(),
    title: `Sent to ${recipient}`,
    amount: amount,
    type: 'debit',
    time: 'Just now',
  };

  walletState = {
    ...walletState,
    walletBalance: walletState.walletBalance - amount,
    transactions: [newTx, ...walletState.transactions],
    isProcessing: false,
  };

  StorageEngine.setItem('walletBalance', walletState.walletBalance);
  StorageEngine.setItem('transactions', walletState.transactions);
  walletListeners.forEach((l) => l());
},
};

const useWalletStore = (selector) => {
  const [, forceUpdate] = useState({});

  useEffect(() => {
    const unsubscribe = () => walletListeners.delete(forceUpdate);
    walletListeners.add(() => forceUpdate({}));
    return () => unsubscribe();
  }, []);

  return selector(walletStore.getState());
};

// 📁 3. MAIN COMPONENT
const App = () => {
  const theme = useWalletStore((state) => state.theme);
  const balance = useWalletStore((state) => state.walletBalance);
  const transactions = useWalletStore((state) => state.transactions);
  const isProcessing = useWalletStore((state) => state.isProcessing);

  const [recipient, setRecipient] = useState('');
  const [amount, setAmount] = useState('');
  const isDark = theme === 'dark';

  const handlePay = () => {
    const numAmount = parseInt(amount, 10);
    if (!recipient.trim() || isNaN(numAmount) || numAmount <= 0) {
      alert('Please enter a valid recipient name & amount! ⚠️');
      return;
    }
    walletStore.sendPayment(recipient, numAmount);
    setRecipient('');
    setAmount('');
  };

  return (

```



```

<SafeAreaView>
  <StatusBar style={isDark ? "light" : "dark"} backgroundColor={isDark ? "#090d16" : "#ffffff"} />
  <SafeAreaView className={`flex-1 px-5 pt-3 ${isDark ? 'bg-slate-950' : 'bg-slate-50'}`}>

    {/* Header */}
    <View className="flex-row justify-between items-center mb-5">
      <View>
        <Text className={`text-2xl font-black ${isDark ? 'text-sky-400' : 'text-sky-600'}`}>
          Smart Wallet 👛
        </Text>
        <Text className={`text-xs ${isDark ? 'text-slate-400' : 'text-slate-500'}`}>
          Persistent Zustand Store
        </Text>
      </View>

      <Pressable
        onPress={() => walletStore.toggleTheme()}
        className={`px-3.5 py-2 rounded-xl border ${isDark ? 'bg-slate-900 border-slate-800' : 'bg-white border-slate-200'}
        >
          <Text className="text-xs font-bold">{isDark ? '🌙 Dark Mode' : '☀️ Light Mode'}</Text>
        </Pressable>
    </View>

    {/* Balance Card */}
    <View className={`p-6 rounded-3xl mb-5 shadow-2xl border ${isDark ? 'bg-slate-900 border-slate-800' : 'bg-white border-slate-200'}`}>
      <Text className={`text-xs font-bold uppercase tracking-wider mb-1 ${isDark ? 'text-slate-400' : 'text-slate-500'}`}>
        Total Available Balance
      </Text>
      <Text className={`text-3xl font-black mb-4 ${isDark ? 'text-emerald-400' : 'text-emerald-600'}`}>
        ₹{balance.toLocaleString('en-IN')}
      </Text>

      <View className="flex-row gap-2">
        <Pressable
          disabled={isProcessing}
          onPress={() => walletStore.addFunds(500)}
          className="flex-1 bg-sky-500 active:bg-sky-600 p-3 rounded-xl items-center"
        >
          <Text className="text-white font-bold text-xs">+ ₹500 Top-up</Text>
        </Pressable>
        <Pressable
          disabled={isProcessing}
          onPress={() => walletStore.addFunds(2000)}
          className="flex-1 bg-indigo-500 active:bg-indigo-600 p-3 rounded-xl items-center"
        >
          <Text className="text-white font-bold text-xs">+ ₹2,000 Top-up</Text>
        </Pressable>
      </View>
    </View>

    {/* Quick Transfer */}
    <View className={`p-4 rounded-2xl mb-5 border ${isDark ? 'bg-slate-900 border-slate-800' : 'bg-white border-slate-200'}`}>
      <Text className={`text-xs font-bold uppercase mb-2 ${isDark ? 'text-slate-300' : 'text-slate-700'}`}>Quick Transfer</Text>
      <View className="flex-row gap-2">
        <TextInput
          value={recipient}
          onChangeText={setRecipient}
          placeholder="Name"
          placeholderTextColor="#64748b"
          className={`flex-1 p-3 rounded-xl border text-xs ${isDark ? 'bg-slate-950 border-slate-800 text-white' : 'bg-slate-50 border-slate-200 text-slate-900'}`}
        />
        <TextInput
          value={amount}
          onChangeText={setAmount}
          placeholder="₹ Amount"
          placeholderTextColor="#64748b"
          keyboardType="numeric"
          className={`w-28 p-3 rounded-xl border text-xs ${isDark ? 'bg-slate-950 border-slate-800 text-white' : 'bg-slate-50 border-slate-200 text-slate-900'}`}
        />
        <Pressable
          disabled={isProcessing}
          onPress={handlePay}
          className={`p-3 rounded-xl justify-center items-center px-4 ${isProcessing ? 'bg-slate-700' : 'bg-emerald-500 active:bg-emerald-600'}`}
        >Pay</Pressable>
      </View>
    </View>
  </SafeAreaView>

```

```

    >
    <Text className="text-slate-950 font-black text-xs">{isProcessing ? '...' : 'Pay'}</Text>
  </Pressable>
</View>
</View>

{/* Recent Activity */}
<Text className={`text-sm font-bold mb-2 ${isDark ? 'text-white' : 'text-slate-900'}`}>Recent Transactions</Text>
<FlatList
  data={transactions}
  keyExtractor={(item) => item.id}
  ItemSeparatorComponent={() => <View className="h-2" />}
  showsVerticalScrollIndicator={false}
  renderItem={({ item }) => (
    <View className={`p-3.5 rounded-2xl flex-row justify-between items-center border ${isDark ? 'bg-slate-900 border-slate-800' : 'bg-white border-slate-200'}`}>
      <View>
        <Text className={`font-bold text-xs ${isDark ? 'text-slate-200' : 'text-slate-800'}`}>{item.title}</Text>
        <Text className="text-slate-400 text-[10px] mt-0.5">{item.time}</Text>
      </View>
      <Text className={`font-black text-xs ${item.type === 'credit' ? 'text-emerald-400' : isDark ? 'text-red-400' : 'text-red-500'}`}>
        {item.type === 'credit' ? '+' : '-'} ₹{item.amount.toLocaleString('en-IN')}
      </Text>
    </View>
  )}
/>

</SafeAreaView>
</SafeAreaProvider>
);
};

export default App;

```

Telugu + English Explanation:

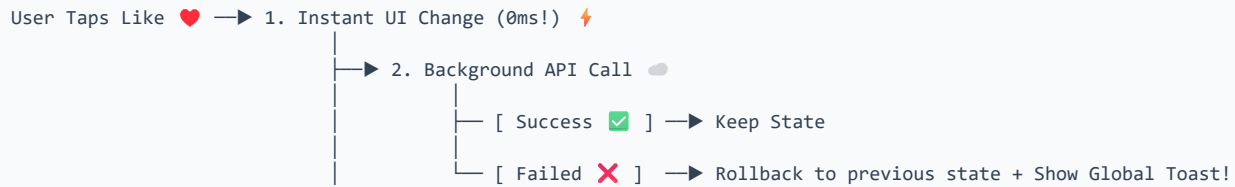
- **persist Middleware:** App close chesina, phone restart chesina mana wallet balance & theme delete avvakunda automatic ga storage lo save aypothundi.
- **Async Global Actions:** `addFunds` or `sendPayment` actions lo direct `async/await` functions vaadathamu.

⚡ Level 6.4: Optimistic UI Updates & Global Notification Toasts

1. Mental Model: What is an "Optimistic UI Update"?

In top-tier mobile apps (Instagram, Twitter, WhatsApp), when you tap like, the heart icon turns red **in 0 milliseconds** without waiting for the server!

If the background network request fails, the app **automatically rolls back** to the previous state and shows a global floating toast notification.



2. Global Toast Manager with Zustand

Any component anywhere in the app can trigger `useToastStore.getState().showToast('Item deleted!', 'error')` without passing props down the tree.

Code Example: Interactive Optimistic Like & Global Toast System

```
import React, { useState } from 'react';
import { View, Text, Pressable, FlatList, ActivityIndicator } from 'react-native';
import { SafeAreaView, SafeAreaView } from 'react-native-safe-area-context';
import { StatusBar } from 'expo-status-bar';

// 📦 1. GLOBAL TOAST MANAGER STORE
let toastState = {
  visible: false,
  message: '',
  type: 'info',
};

const toastListeners = new Set();

const toastStore = {
  getState: () => toastState,
  showToast: (message, type = 'info', duration = 2500) => {
    toastState = { visible: true, message, type };
    toastListeners.forEach((l) => l());

    setTimeout(() => {
      toastState = { ...toastState, visible: false };
      toastListeners.forEach((l) => l());
    }, duration);
  },
  hideToast: () => {
    toastState = { ...toastState, visible: false };
    toastListeners.forEach((l) => l());
  },
};

const useToastStore = (selector) => {
  const [, forceUpdate] = useState({});
  React.useEffect(() => {
    const unsubscribe = () => toastListeners.delete(forceUpdate);
    toastListeners.add(() => forceUpdate({}));
    return () => unsubscribe();
  }, []);
  return selector(toastStore.getState());
};

// 📦 2. POSTS STORE (With Optimistic Updates & Rollbacks)
const INITIAL_POSTS = [
  { id: '1', title: 'Zero Latency with Zustand & NativeWind', author: 'Kiran', likes: 142, isLiked: false },
  { id: '2', title: 'Why FlashList beats FlatList in 2026', author: 'Alex', likes: 89, isLiked: false },
  { id: '3', title: 'Hardware Keystores vs Plain Storage', author: 'Sara', likes: 215, isLiked: true },
];

let feedState = {
  posts: INITIAL_POSTS,
  simulateNetworkFailure: false,
};

const feedListeners = new Set();

const feedStore = {
  getState: () => feedState,

  toggleNetworkFailureSimulation: () => {
    feedState = { ...feedState, simulateNetworkFailure: !feedState.simulateNetworkFailure };
    feedListeners.forEach((l) => l());
  },

  toggleLikePost: async (postId) => {
    const post = feedState.posts.find((p) => p.id === postId);
    if (!post) return;

    const previousIsLiked = post.isLiked;
    const previousLikes = post.likes;

    // 1. OPTIMISTIC UPDATE: Instant (0ms)
```

```

feedState = {
  ...feedState,
  posts: feedState.posts.map((p) =>
    p.id === postId
      ? { ...p, isLiked: !p.isLiked, likes: p.isLiked ? p.likes - 1 : p.likes + 1 }
      : p
  ),
};
feedListeners.forEach((l) => l());

// 2. Background API Simulation
try {
  await new Promise((res) => setTimeout(res, 800));

  if (feedState.simulateNetworkFailure) {
    throw new Error('500 Server Error: Failed to sync like status');
  }

  toastStore.showToast(!previousIsLiked ? 'Post Liked! ❤️' : 'Post Unliked ❤️', 'success');
} catch (err) {
  // 3. ROLLBACK
  feedState = {
    ...feedState,
    posts: feedState.posts.map((p) =>
      p.id === postId ? { ...p, isLiked: previousIsLiked, likes: previousLikes } : p
    ),
  };
  feedListeners.forEach((l) => l());
  toastStore.showToast('Network error: Like rolled back! ⚠️', 'error');
}
},
};

const useFeedStore = (selector) => {
  const [, forceUpdate] = useState({});
  React.useEffect(() => {
    const unsubscribe = () => feedListeners.delete(forceUpdate);
    feedListeners.add(() => forceUpdate({}));
    return () => unsubscribe();
  }, []);
  return selector(feedStore.getState());
};

// 🍞 3. FLOATING GLOBAL TOAST COMPONENT
const GlobalToast = () => {
  const toast = useToastStore((state) => state);
  if (!toast.visible) return null;

  const bgStyle =
    toast.type === 'success' ? 'bg-emerald-500 text-slate-950 border-emerald-400' :
    toast.type === 'error' ? 'bg-red-500 text-white border-red-400' :
    'bg-sky-500 text-white border-sky-400';

  return (
    <View className="absolute top-12 left-5 right-5 z-50 shadow-2xl">
      <View className={`py-3 px-4 rounded-2xl flex-row items-center justify-between border ${bgStyle}`}>
        <Text className="font-extrabold text-xs flex-1 pr-2">{toast.message}</Text>
        <Pressable onPress={() => toastStore.hideToast()}>
          <Text className="font-black text-xs">X</Text>
        </Pressable>
      </View>
    </View>
  );
};

// 🏠 4. MAIN APP
const App = () => {
  const posts = useFeedStore((state) => state.posts);
  const simulateFailure = useFeedStore((state) => state.simulateNetworkFailure);

  return (
    <SafeAreaProvider>
      <StatusBar style="light" backgroundColor="#090d16" />

```

```

<SafeAreaView className="flex-1 bg-slate-950 px-5 pt-3 justify-between pb-6">
  <GlobalToast />

  <View className="flex-1">
    <View className="mb-4">
      <Text className="text-2xl font-black text-sky-400">Optimistic UI ⚡ </Text>
      <Text className="text-slate-400 text-xs">Zero-latency feedback with automatic error rollback</Text>
    </View>

    <Pressable
      onPress={() => feedStore.toggleNetworkFailureSimulation()}
      className={`p-3.5 rounded-2xl mb-4 border flex-row justify-between items-center ${
        simulateFailure ? 'bg-red-500/20 border-red-500/40' : 'bg-slate-900 border-slate-800'
      }`}
    >
      <View>
        <Text className="text-white font-bold text-xs">Simulate Bad Network / 500 Error</Text>
        <Text className="text-slate-400 text-[10px]">
          {simulateFailure ? 'Enabled (Likes will fail and rollback)' : 'Disabled (Likes will succeed)'}
        </Text>
      </View>
      <View className={`w-5 h-5 rounded-full ${simulateFailure ? 'bg-red-500' : 'bg-slate-700'}`} />
    </Pressable>

    <Text className="text-white font-bold text-sm mb-3">Community Tech Feed</Text>
    <FlatList
      data={posts}
      keyExtractor={(item) => item.id}
      ItemSeparatorComponent={() => <View className="h-3" />}
      renderItem={({ item }) => (
        <View className="bg-slate-900 border border-slate-800 p-4 rounded-2xl">
          <Text className="text-white font-bold text-sm mb-1">{item.title}</Text>
          <Text className="text-slate-400 text-xs mb-3">Posted by @{item.author.toLowerCase()}</Text>

          <View className="flex-row justify-between items-center border-t border-slate-800/80 pt-3">
            <Text className="text-slate-400 text-xs font-semibold">{item.likes} likes</Text>
            <Pressable
              onPress={() => feedStore.toggleLikePost(item.id)}
              className={`px-3.5 py-1.5 rounded-xl flex-row items-center gap-1.5 border ${
                item.isLiked ? 'bg-red-500/20 border-red-500/40' : 'bg-slate-950 border-slate-800'
              }`}
            >
              <Text className="text-sm">{item.isLiked ? '❤️' : '💜'}</Text>
              <Text className={`text-xs font-bold ${item.isLiked ? 'text-red-400' : 'text-slate-400'}`}>
                {item.isLiked ? 'Liked' : 'Like'}
              </Text>
            </Pressable>
          </View>
        </View>
      )}
    />
  </View>
</SafeAreaView>
</SafeAreaProvider>
);
};

export default App;

```

🧑 Telugu + English Explanation:

- **Optimistic UI Pattern:** User heart icon tap cheyagane 0 milliseconds lo heart ni red chestham.
- **Rollback Mechanism:** API call fail ayithe previous state snapshot theskuni state ni revert (rollback) chesi, top lo global Toast chupistham.